

Experimental Report on Multi-Target Stockout Forecasting Using the FreshRetailNet-50K Dataset

Ha Thuc Khuong Ninh¹

FPT University, Ho Chi Minh City, Vietnam
hathuckhuongninh@gmail.com

Abstract. This report presents a workflow for building a stockout forecasting model in fresh food retail using the FreshRetailNet-50K dataset. Because the original dataset is large, the experiment focuses on the **City ID 3** subset, which contains 238,320 records, 101 stores, and 173 products. The workflow consists of three main stages: data preprocessing, exploratory data analysis (EDA), and training a multi-output XGBoost model. From the `hours_stock_status` field, the system extracts 16 binary labels corresponding to operating hours from 6:00 to 21:00. A `MultiOutputClassifier` with an `XGBClassifier` core is trained on 83 days of data and tested on the final 7 days. After optimizing a separate prediction threshold for each hour using the Precision–Recall curve, the model achieves an average Accuracy of 72.22%, Precision of 41.47%, Recall of 62.34%, and F1-Score of 49.43% on 18,536 test samples. The results show that the model can detect a meaningful portion of stockout cases, although it still exhibits a trade-off between Recall and Precision because of the class imbalance inherent in stockout forecasting.

1 Introduction

Inventory management in fresh food retail is challenging because products have short shelf lives, demand changes rapidly throughout the day, and replenishment is constrained by logistics. When a product is out of stock, the sales system often records zero sales. This zero value does not necessarily indicate low demand; it may instead result from customers being unable to purchase the product. This phenomenon creates censored sales data and causes traditional demand forecasting models to underestimate true demand.

The objective of this report is to build an experimental workflow that forecasts stockout states at multiple checkpoints within a day. Instead of predicting a single daily label, the model simultaneously predicts 16 binary states across the operating window from 6:00 to 21:00. This approach helps identify periods that are more likely to experience shortages and supports more time-specific replenishment planning.

The experiment is implemented across three notebooks: `uploads/DataPreprocessing.ipynb` for data preprocessing, `prism-uploads/EDA.ipynb` for exploratory analysis, and

prism-uploads/XGBoost_Model.ipynb for model construction and evaluation.

2 Data and Preprocessing

2.1 Data Source

The data are taken from FreshRetailNet-50K, a fresh food retail dataset with hourly inventory-state annotations. Because the full training dataset contains approximately 4.5 million rows, the experiment filters only records from **City ID 3**. After filtering, the subset contains 238,320 rows and 19 initial attributes.

Table 1. Summary of the preprocessed dataset

Item	Value
Records for City ID 3	238,320
Initial attributes	19
Attributes after dropping hierarchy columns	14
Unique stores	101
Unique products	173
Missing values	0
Duplicate rows	0
Output dataset file	freshretailnet_city_03_dataset.csv

2.2 Preprocessing Steps

The preprocessing workflow focuses on producing a clean, consistent table suitable for machine learning:

- **Regional filtering:** Only records with `city_id = 3` are retained to reduce computational cost while preserving a multi-store and multi-product structure.
- **Dropping hierarchy attributes:** The columns `management_group_id`, `first_category_id`, `second_category_id`, `third_category_id`, and `city_id` are removed because they are not directly used by the model in this experiment.
- **Data-quality checking:** The notebook confirms that there are no missing values and no duplicate rows after excluding list-type columns from the duplicate check.
- **Identifier normalization:** `store_id` and `product_id` are converted into readable string forms such as `Store_70` and `Product_157` for EDA, then label-encoded before model training.

- **Hourly stock-state parsing:** The `hours_stock_status` field is converted from a list string into a 24-element binary vector using `ast.literal_eval`. The model uses positions 6 through 21 to create 16 target labels for operating hours.

3 Exploratory Data Analysis

3.1 Descriptive Statistics

The City ID 3 data show a clearly skewed distribution of demand and inventory states. The average `sale_amount` is 0.679, the median is 0.600, and the maximum value is 10.000. The variable `stock_hour6_22_cnt` has a mean of 3.610 and a standard deviation of 5.492, indicating that many records have no inventory disruption while a substantial subset still experiences long stockout durations.

Regarding operating context, `holiday_flag` accounts for 34.44% of records and `activity_flag` accounts for 33.72%. The average `discount` is 0.915, which corresponds to an approximate 8.5% discount if 1.000 is interpreted as no discount. Weather variables also provide meaningful contextual variation, including precipitation, average temperature, average humidity, and average wind level.

3.2 Market Structure and Stockouts

The EDA results show a long-tail structure across both stores and products. `Store_429` contains 5,940 records, while several smaller stores contain only a few hundred records. Similarly, `Product_834` contains 9,090 records, much higher than products that appear infrequently. This implies that the model must learn behavior from entities with very different observation frequencies.

According to the EDA notebook, more than half of the stores have an average stockout duration exceeding 4 hours per day. The hourly stockout curve also varies clearly throughout the day: shortage states rise in some operating windows and then decline as replenishment activities take place. Figure 1 illustrates four main EDA perspectives: hourly stockout rate, sales-volume segmentation, product distribution by average stockout hours, and store proportion by average stockout level.

4 Model Construction

4.1 Problem Formulation

For each record corresponding to a store-product pair on day t , the objective is to predict the stockout-state vector across 16 operating hours:

$$\hat{Y}_t = [\hat{y}_{t,6}, \hat{y}_{t,7}, \dots, \hat{y}_{t,21}] \in \{0, 1\}^{16}. \quad (1)$$

Here, $\hat{y}_{t,h} = 1$ indicates that the model predicts a stockout at hour h , while $\hat{y}_{t,h} = 0$ indicates no stockout. This is a multi-label classification task over intraday time points because a single record may contain stockouts at multiple hours.

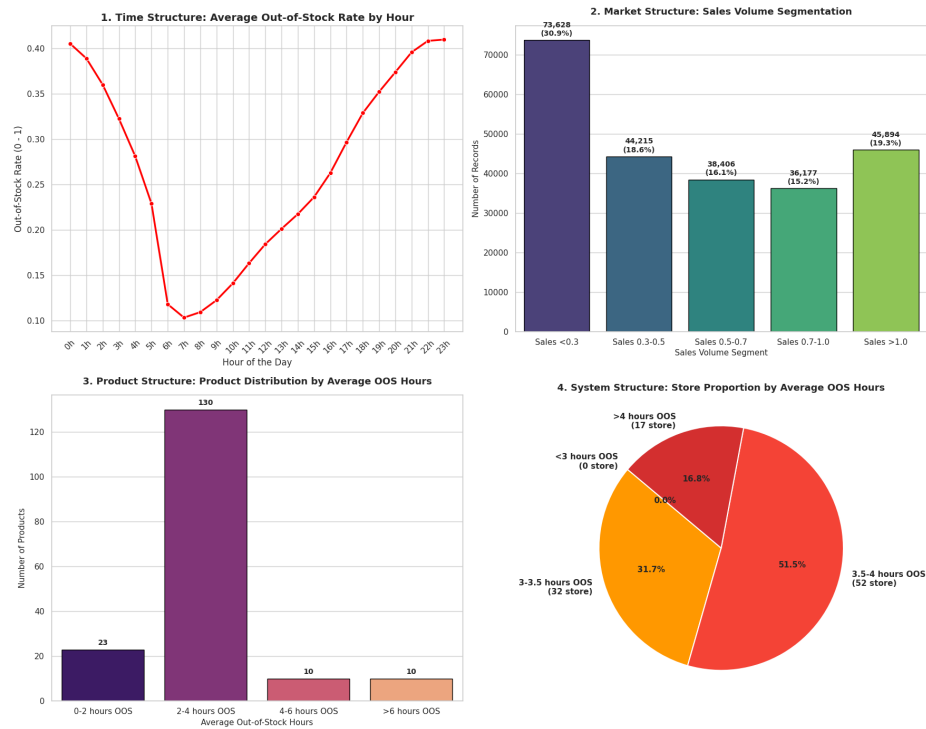


Fig. 1. EDA overview for the City ID 3 dataset.

4.2 Input Features

The model notebook uses 9 input features:

`store_encoded`, `product_encoded`, `discount`, `holiday_flag`, `activity_flag`, `precpt`, `avg_temperature`, `avg_humidity`, and `avg_wind_level`.

The two features `store_encoded` and `product_encoded` represent label-encoded entity identifiers. The remaining seven features describe promotions, holidays, campaign activity, and weather conditions.

4.3 Training Setup

The data are sorted by date to prevent temporal leakage. The training set contains 83 days and 219,784 samples, while the test set contains the final 7 days and 18,536 samples. The model uses `MultiOutputClassifier` to train 16 parallel classifiers, one for each hour from 6:00 to 21:00. The base classifier is `XGBClassifier` with the following main parameters: `n_estimators=50`, `max_depth=5`, `learning_rate=0.1`, `random_state=42`, and `eval_metric=logloss`.

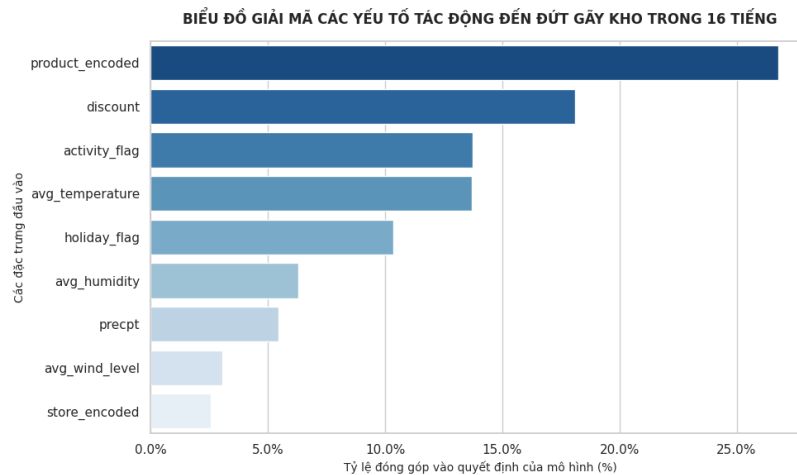


Fig. 2. Average feature contribution across the 16 parallel XGBoost models.

4.4 Prediction-Threshold Optimization

Because stockout forecasting is usually class-imbalanced, using the default threshold of 0.5 may cause the model to miss many stockout cases. Therefore, the notebook computes a Precision–Recall curve for each hour and selects the threshold that maximizes F1-Score. Table 2 shows the 16 optimized thresholds used during evaluation.

Table 2. Optimized prediction thresholds by operating hour

Hour	6h	7h	8h	9h	10h	11h	12h	13h
Threshold	0.196	0.219	0.237	0.233	0.175	0.196	0.205	0.206
Hour	14h	15h	16h	17h	18h	19h	20h	21h
Threshold	0.219	0.239	0.251	0.279	0.299	0.328	0.337	0.345

5 Experimental Results

5.1 Aggregate Evaluation Metrics

The evaluation on 18,536 test samples shows that the model achieves an average Accuracy of 72.22% across the 16 hourly targets. Precision reaches 41.47%, while Recall reaches 62.34%. This indicates that the model prioritizes detecting stockout cases rather than predicting only highly certain cases, which is useful when the operational goal is to reduce missed stockout events.

Table 3. Evaluation results of the multi-output XGBoost model

Metric	Value
Test samples	18,536
Average Accuracy	72.22%
Average Precision	41.47%
Average Recall	62.34%
Average F1-Score	49.43%

5.2 Confusion-Matrix and Missed-Case Analysis

Figure 3 shows the confusion matrix for each hourly target. The 4x4 grid makes it possible to observe correct and incorrect predictions at each time point. Hours with lower stockout rates may obtain higher Accuracy, but Accuracy alone can hide missed cases in the minority class.

To focus on stockout detection, Figure 4 separates the proportion of actual stockout cases that are captured from the proportion that is missed for each hour. This perspective is more informative than Accuracy when the cost of missing a stockout is higher than the cost of a false alarm. The result shows that some hours still have a notable missed-case ratio, suggesting that the model can be improved with historical features or additional inventory and logistics information.

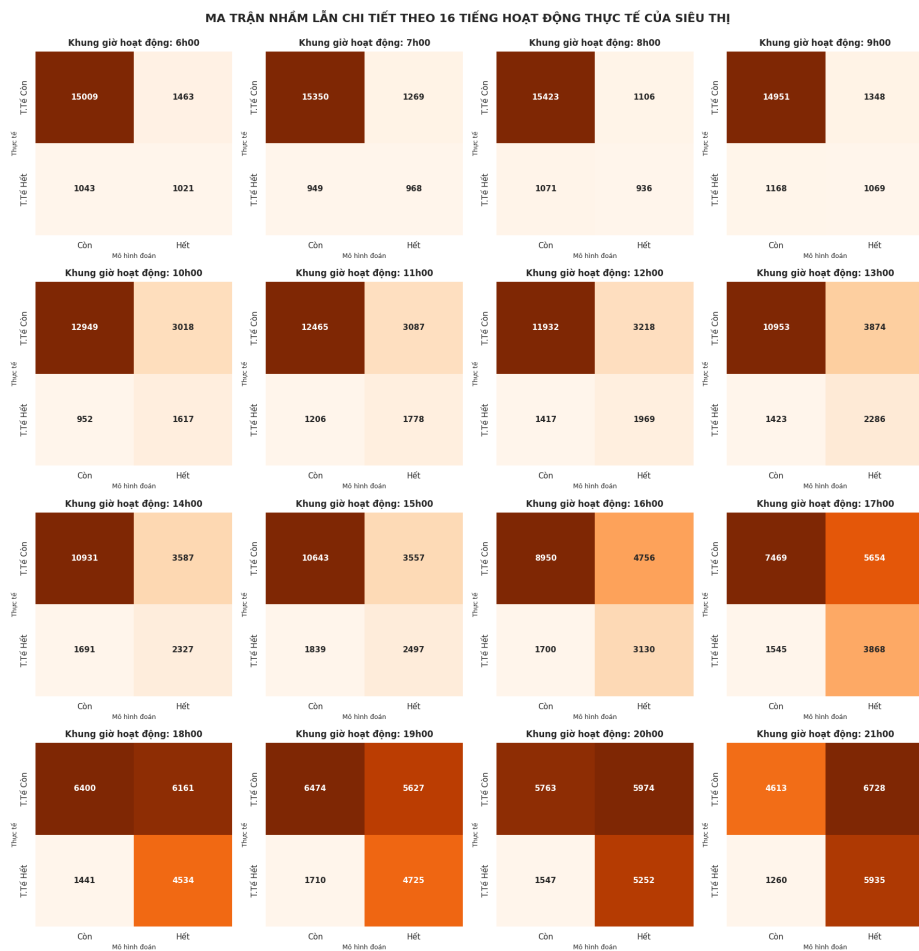


Fig. 3. Confusion matrices for the 16 operating-hour targets.

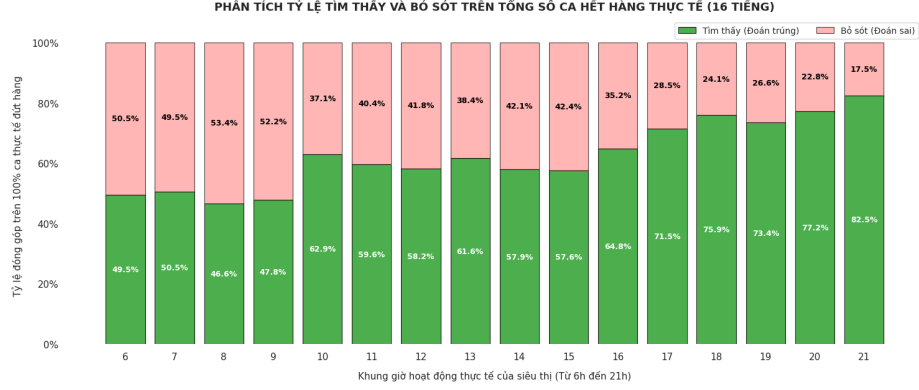


Fig. 4. Detected and missed proportions of actual stockout cases by hour.

5.3 Discussion

The experimental results show that multi-output XGBoost is a suitable baseline for hourly stockout forecasting because it trains quickly, is relatively interpretable, and handles nonlinear relationships among store identity, product identity, promotions, and weather. However, the low Precision indicates that the model still produces many false-positive alerts. In operations, this may be acceptable if the main objective is early warning, but it should be controlled to avoid overloading replenishment teams.

A key limitation is that the current model does not directly use lagged time-series features such as sales from previous days, previous inventory states of the same product, or replenishment schedule information. These features could improve Precision and F1-Score because they better capture consumption trends and logistics cycles than identifiers, promotions, and weather alone.

6 Conclusion and Future Work

This report presents a complete experimental workflow from preprocessing to model evaluation for multi-target stockout forecasting on the FreshRetailNet-50K dataset. The City ID 3 subset, containing 238,320 records, is cleaned, analyzed, and used to train a multi-output XGBoost model for 16 operating-hour targets. After hour-specific threshold optimization, the model achieves a Recall of 62.34% and an F1-Score of 49.43%, indicating meaningful stockout-detection capability while leaving room for improvement.

Future work can extend this report in three directions. First, lagged features at the store-product level can be added so that the model captures short-term consumption trends. Second, XGBoost can be compared with time-series and deep learning models such as TimesNet, PatchTST, and Temporal Fusion Transformer. Third, operational error costs can be incorporated so that prediction thresholds are selected not only by F1-Score but also by business objectives.

References

1. Zeng, A., Chen, M., Zhang, L., Xu, Q.: Are Transformers Effective for Time Series Forecasting? AAAI **37**(9), 11121–11128 (2023).
2. Nie, Y., Nguyen, N. H., Sinthong, P., Kalagnanam, J.: A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. ICLR (2023).
3. Dingdong-Inc: FreshRetailNet-50K: A Stockout-Annotated Censored Demand Dataset for Latent Demand Recovery and Forecasting in Fresh Retail. arXiv:2505.16319 (2025).
4. Dingdong-Inc repository: Fresh Retail 50K Baseline Infrastructure. <https://github.com/Dingdong-Inc/frn-50k-baseline>.